LESSON 01 · LOGGING · 50 MIN

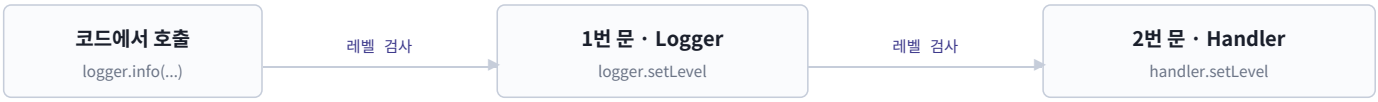
1교시 Logging 기초 — 문이 두 개인 이유

로그가 안 보이는 이유는 거의 언제나 레벨입니다. 레코드는 로거의 레벨을 통과하고 다시 핸들러의 레벨을 통과해야 출력됩니다. 두 관문을 직접 만들어 보고, 어느 쪽에서 막히는지 눈으로 확인합니다.

개념 15분 · 실습 25분 · 정리와 질문 10분

OVERVIEW

레코드가 지나가는 두 개의 문



> 두 문을 **모두** 통과해야 출력됩니다. 로거에서 막히면 레코드 자체가 만들어지지 않으므로, 핸들러를 아무리 DEBUG로 낮춰도 아무 일도 일어나지 않습니다.

REFERENCE

이 시간에 쓰는 명령

명령	하는 일	언제 쓰나
`logging.getLogger(__name__)`	모듈 이름으로 로거를 얻는다	모든 모듈의 첫 줄 · 사실상 관용구
`logger.setLevel(...)`	1번 문 · 이 로거가 만들 최소 레벨	보통 DEBUG로 열어 두고 핸들러로 조절
`handler.setLevel(...)`	2번 문 · 이 출력구가 받을 최소 레벨	콘솔은 INFO, 파일은 DEBUG 처럼
`logging.StreamHandler()`	콘솔(표준에러)로 내보낸다	개발 중 눈으로 볼 때
`logging.FileHandler(...)`	파일로 남긴다	나중에 뒤져 봐야 할 때
`logging.Formatter(fmt)`	한 줄을 어떤 모양으로 찍을지	시각·레벨·위치를 넣는다
`logging.basicConfig(...)`	루트 로거를 한 방에 설정	작은 스크립트 · 실습 초반
`logger.debug("x=%s", x)`	값을 나중에 끼워 넣는 형식	f-string 대신 쓰는 이유는 아래 STEP 08

FILES

실습용 파일

orders.py

```
def apply_discount(price, rate):
    if not 0 <= rate <= 1:
        raise ValueError("rate must be between 0 and 1")
    return round(price * (1 - rate))


def checkout(cart, rate=0.0):
    total = sum(p * q for _, p, q in cart)
    return apply_discount(total, rate)
```

HANDS-ON

실습 순서

STEP 01 폴더와 파일 준비

이번 6교시 내내 쓸 작업 폴더를 만듭니다.

```
mkdir logging-practice && cd logging-practice
python --version
# 위 '실습용 파일'의 orders.py 를 이 폴더에 만듭니다
```

확인 포인트

- [ ] Python 3.11 이상이 출력된다
- [ ] orders.py 가 폴더에 있다

> 가상환경을 쓰신다면 **python -m venv .venv** 후 활성화해 두세요. 3교시에서 pytest를 설치할 때 편합니다.

STEP 02 print의 한계를 먼저 겪어 보기

왜 로깅으로 갈아타야 하는지 몸으로 확인합니다.

```
# main.py 를 만들고 아래 내용을 넣습니다
from orders import checkout

cart = [("book", 12000, 2)]
print("계산 시작")
print("결과:", checkout(cart, 0.1))
print("계산 끝")

python main.py
```

확인 포인트

- [ ] 세 줄이 출력된다
- [ ] 이 출력을 끄려면 코드를 직접 지우는 수밖에 없다
- [ ] 언제 찍혔는지, 어느 파일에서 찍혔는지 알 수 없다

> print는 **끄고 켤 수 없고, 시각과 위치가 남지 않고, 심각도 구분이 없습니다.** 이 세 가지가 로깅으로 넘어가는 이유의 전부입니다.

STEP 03 basicConfig 로 첫 로그 남기기

가장 짧은 형태로 로깅을 시작합니다.

```
# main.py 를 아래로 바꿉니다
import logging
from orders import checkout

logging.basicConfig(level=logging.INFO)

cart = [("book", 12000, 2)]
logging.info("계산 시작")
logging.info("결과: %s", checkout(cart, 0.1))
logging.debug("장바구니 상세: %s", cart)

python main.py
```

확인 포인트

- [ ] INFO 두 줄이 출력된다
- [ ] debug 한 줄은 출력되지 않는다 — level=INFO 이기 때문
- [ ] 출력 앞에 INFO:root: 가 붙는다

> **root** 라고 찍히는 것에 주목하세요. 아직 내 로거를 만들지 않아 루트 로거를 그대로 쓰고 있다는 뜻입니다.

STEP 04 레벨을 바꿔 가며 출력량 비교

레벨 하나로 출력이 어떻게 줄어드는지 봅니다.

```
# main.py 에 다섯 줄을 모두 넣어 봅니다
logging.debug("가장 상세한 내부 값")
logging.info("정상 흐름 기록")
logging.warning("이상하지만 계속 진행")
logging.error("실패했지만 프로그램은 살아 있음")
logging.critical("더 진행할 수 없음")

# basicConfig 의 level 을 DEBUG → WARNING 으로 바꿔 가며 실행
python main.py
```

확인 포인트

- [ ] DEBUG면 5줄, INFO면 4줄, WARNING이면 3줄이 나온다
- [ ] 코드는 한 글자도 고치지 않았는데 출력량이 달라진다

> 이것이 print가 못 하는 일입니다. **개발 중에는 DEBUG, 운영에서는 WARNING** 처럼 배포 환경에 따라 설정만 바꾸면 됩니다.

STEP 05 format 으로 정보 붙이기

한 줄에 시각·레벨·위치를 담습니다.

```
logging.basicConfig(
    level=logging.DEBUG,
    format="%(asctime)s %(levelname)-8s %(name)s %(filename)s:%(lineno)d %(message)s",
)

python main.py
```

확인 포인트

- [ ] 앞에 날짜와 시각이 붙는다
- [ ] levelname 이 8칸으로 정렬되어 읽기 쉬워진다
- [ ] 어느 파일 몇 번째 줄에서 찍혔는지 보인다

> **filename:lineno 는 실무에서 가장 값어치가 큽니다.** 오류 로그를 보고 바로 그 줄로 갈 수 있으니까요.

STEP 06 내 로거 만들기 — getLogger(__name__)

루트 로거를 그만 쓰고 모듈별 로거로 옮깁니다.

```
# orders.py 맨 위에 추가
import logging
logger = logging.getLogger(__name__)

# apply_discount 안에 추가
logger.debug("할인 적용 price=%s rate=%s", price, rate)

python main.py
```

확인 포인트

- [ ] 로그의 name 자리가 root 가 아니라 orders 로 바뀐다
- [ ] 어느 모듈에서 나온 로그인지 구분된다

> **__name__** 을 쓰면 로거 이름이 모듈 경로를 그대로 따라갑니다. 2교시의 계층 구조가 여기서 시작됩니다.

STEP 07 Handler 를 직접 붙이기 — 오늘의 핵심 (1)

basicConfig 를 걷어내고 콘솔·파일을 따로 구성합니다.

```
# log_setup.py 를 새로 만듭니다
import logging

def setup():
    logger = logging.getLogger("app")
    logger.setLevel(logging.DEBUG)

    fmt = logging.Formatter(
        "%(asctime)s %(levelname)-8s %(name)s %(filename)s:%(lineno)d %(message)s")

    console = logging.StreamHandler()
    console.setLevel(logging.INFO)
    console.setFormatter(fmt)

    file = logging.FileHandler("app.log", encoding="utf-8")
    file.setLevel(logging.DEBUG)
    file.setFormatter(fmt)

    logger.addHandler(console)
    logger.addHandler(file)
    return logger
```

확인 포인트

- [ ] 콘솔은 INFO, 파일은 DEBUG 로 서로 다른 레벨이다
- [ ] FileHandler 에 encoding="utf-8" 을 넣었다

> **encoding="utf-8" 은 빼먹지 마세요.** 윈도우에서는 기본 인코딩 때문에 한글 로그가 깨지거나 오류가 납니다.

STEP 08 두 개의 문 확인하기 — 오늘의 핵심 (2)

로거 레벨과 핸들러 레벨 중 무엇이 먼저 막는지 실험합니다.

```
# main.py 에서 setup() 을 쓰도록 바꾸고 실행
from log_setup import setup
logger = setup()
logger.debug("디버그"); logger.info("정보"); logger.error("오류")
python main.py && cat app.log

# 실험 A · 핸들러만 낮춰 보기
# console.setLevel(logging.DEBUG) 로 바꾸고 다시 실행

# 실험 B · 로거를 올려 보기
# logger.setLevel(logging.WARNING) 으로 바꾸고 다시 실행
```

확인 포인트

- [ ] 기본 상태: 콘솔에는 2줄(info, error), 파일에는 3줄이 남는다
- [ ] 실험 A: 콘솔에도 3줄이 나온다
- [ ] 실험 B: 핸들러가 DEBUG여도 콘솔·파일 모두 error 한 줄만 남는다

> 실험 B가 이 시간의 결론입니다. 로거에서 막히면 레코드가 아예 만들어지지 않아 핸들러까지 가지도 않습니다.
> 그래서 **로거는 DEBUG로 열어 두고 핸들러 레벨로 조절**하는 구성이 안전합니다.

STEP 09 print 를 모두 로그로 교체하고 정리

배운 것을 코드 전체에 적용합니다.

```
# 남은 print 를 전부 logger 로 바꿉니다
# 나쁜 예 (f-string)
logger.debug(f"장바구니: {cart}")
# 좋은 예 (지연 포맷)
logger.debug("장바구니: %s", cart)

python main.py
```

확인 포인트

- [ ] 코드에 print 가 하나도 남아 있지 않다
- [ ] app.log 파일이 만들어져 있고 내용이 읽힌다

> **%s 형식을 쓰는 이유** · 레벨에 걸려 출력되지 않을 로그는 문자열을 만들지도 않습니다. f-string 은 레벨과 무관하게 항상 문자열을 먼저 만들기 때문에 DEBUG 로그가 많을수록 손해입니다.

TROUBLESHOOTING

자주 만나는 오류

증상	원인	해결
로그가 하나도 안 나온다	레벨이 높거나 핸들러가 없음	logger.getEffectiveLevel() 과 logger.handlers 를 출력해 확인
basicConfig 를 고쳐도 반영이 안 됨	이미 루트에 핸들러가 붙어 한 번 무시됨	basicConfig(..., force=True) 사용 (Python 3.8+)
같은 줄이 두 번 찍힌다	핸들러를 중복으로 추가함	setup() 을 여러 번 호출하지 않기 · 2교시에서 자세히 다룹니다
한글이 깨지거나 UnicodeEncodeError	파일 인코딩 기본값 (윈도우)	FileHandler(..., encoding="utf-8")
app.log 파일이 안 생긴다	경로 문제 또는 레벨에서 전부 걸림	pwd 로 실행 위치 확인 · 핸들러 레벨을 DEBUG로 낮춰 확인

REVIEW

복습 체크리스트 — 답할 수 있으면 통과

- [ ] print 대신 logging 을 쓰는 이유를 세 가지 말해 보세요.
- [ ] 로거 레벨이 WARNING인데 핸들러가 DEBUG면 debug 로그는 나올까요?
- [ ] getLogger(__name__) 이 남기는 name 값은 무엇이 되나요?
- [ ] 콘솔에는 요약만, 파일에는 상세까지 남기려면 무엇을 다르게 설정하나요?
- [ ] logger.debug(f"{x}") 대신 logger.debug("%s", x) 를 권하는 이유는?

한 줄 정리

레벨은 출력을 줄이는 장치가 아니라, 같은 코드를 개발과 운영에서 다르게 동작시키는 손잡이입니다.

다음 시간 → 2교시 — 로그가 두 번 찍히고, 라이브러리 로그가 쏟아질 때

WORKSHEET

막힌 STEP 번호 · 화면에 나온 오류 메시지 · 물어볼 것